

The structure of a Svelte file

Every `.svelte` file has three sections. You'll work in all three this semester.

```
None
<script>
  // logic and data lives here
</script>

<!-- markup lives here -->

<style>
  /* styles live here */
</style>
```

SVG basics

SVG is a way of describing shapes that the browser can draw. You write SVG tags directly in the markup section of your Svelte file, just like HTML.

The coordinate system

The most important thing to internalize: `0, 0` is the **top-left** corner. X increases to the right. Y increases **downward**.

```
None
<svg viewBox="0 0 600 400" width="600" height="400" style="border:
1px solid #ccc">
</svg>
```

`viewBox="0 0 600 400"` defines your canvas: 600 units wide, 400 units tall.

SVG primitives

None

```
<svg viewBox="0 0 600 400" width="600" height="400" style="border: 1px solid #ccc">

  <!-- rectangle: x and y are the top-left corner -->
  <rect x="50" y="50" width="100" height="60" fill="steelblue" />

  <!-- circle: cx and cy are the center -->
  <circle cx="300" cy="100" r="40" fill="tomato" />

  <!-- line -->
  <line x1="50" y1="200" x2="400" y2="200" stroke="black"
stroke-width="2" />

  <!-- text -->
  <text x="50" y="300" font-size="20">Hello, SVG</text>

</svg>
```

From hardcoded to data-driven

Step 1: Hardcoded shapes

A column chart is just rectangles. Here's one with three columns, every value typed by hand.

None

```
<script>
</script>

<svg viewBox="0 0 600 400" width="600" height="400" style="border: 1px solid #ccc">
  <rect x="50" y="320" width="60" height="80" fill="steelblue" />
  <rect x="150" y="250" width="60" height="150" fill="steelblue" />
```

```
<rect x="250" y="200" width="60" height="200" fill="steelblue" />
</svg>
```

Notice: `y = 400 - height`. Because `y=0` is at the top, columns grow upward by starting lower on the canvas. This is the coordinate system gotcha to keep in mind.

Step 2: Data in the script block

Move your values into an array. You can now reference them directly by index instead of typing the numbers twice.

```
None
<script>
  const data = [80, 150, 200]
</script>

<svg viewBox="0 0 600 400" width="600" height="400" style="border:
1px solid #ccc">
  <rect x="50" y={400 - data[0]} width="60" height={data[0]}
fill="steelblue" />
  <rect x="150" y={400 - data[1]} width="60" height={data[1]}
fill="steelblue" />
  <rect x="250" y={400 - data[2]} width="60" height={data[2]}
fill="steelblue" />
</svg>
```

Change a value in the array and both `y` and `height` update together. But you still have one `rect` per data point. What happens when you have 50 columns?

Step 3: `{#each}` loop

Replace the three hardcoded `rects` with a single loop. Same output, one element in the template.

None

```
<script>
  const data = [80, 150, 200]

  const width = 600
  const height = 400
</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  {#each data as value, i}
    <rect
      x={50 + i * 100}
      y={height - value}
      width="60"
      height={value}
      fill="steelblue"
    />
  {/each}
</svg>
```

What each part does:

- **data** — the array being looped over
- **value** — one item from the array (one row of data)
- **i** — the index (0, 1, 2...), used to space columns horizontally
- **height={value}** — the channel: column height encodes the data value
- **y={height - value}** — positions the column so it grows upward from the bottom

Try adding and removing values from the array. The chart updates automatically.

Step 4: Objects instead of plain numbers

Real data is rarely a flat array of numbers. Each row is usually an object with named columns.

None

```
<script>
```

```

const data = [
  { city: "New York",    temp: 84 },
  { city: "Philadelphia", temp: 91 },
  { city: "Chicago",     temp: 78 },
  { city: "Los Angeles",  temp: 72 },
  { city: "Houston",     temp: 96 }
]

const width = 600
const height = 400
</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  {#each data as d, i}
    <rect
      x={50 + i * 100}
      y={height - d.temp}
      width="60"
      height={d.temp}
      fill="steelblue"
    />
  {/each}
</svg>

```

`d.temp` is the channel and is the column driving the height. `d.city` isn't being used yet. Where would you put it? How would you adjust your chart's baseline to accommodate labels?

Step 5: A second channel

Encode a second variable using color.

None

```

<script>
  const data = [

```

```

    { city: "New York",      temp: 84, region: "Northeast" },
    { city: "Philadelphia", temp: 91, region: "Northeast" },
    { city: "Chicago",      temp: 78, region: "Midwest"   },
    { city: "Los Angeles",  temp: 72, region: "West"      },
    { city: "Houston",      temp: 96, region: "South"     }
  ]

  const regionColors = {
    Northeast: "steelblue",
    Midwest:   "seagreen",
    West:      "tomato",
    South:     "goldenrod"
  }

  const width = 600
  const height = 400
</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  {#each data as d, i}
    <rect
      x={50 + i * 100}
      y={height - d.temp}
      width="60"
      height={d.temp}
      fill={regionColors[d.region]}
    />
  {/each}
</svg>

```

Two channels: height encodes temperature, color encodes region. Ask yourself: what is the mark, what are the channels, what column drives each one?

More chart types

Bar chart

Swap the role of x and y. Width becomes the channel instead of height, so the y-gotcha goes away.

None

```
<script>
  const data = [
    { city: "New York",    temp: 84 },
    { city: "Philadelphia", temp: 91 },
    { city: "Chicago",     temp: 78 },
    { city: "Los Angeles", temp: 72 },
    { city: "Houston",     temp: 96 }
  ]

  const width = 600
  const height = 400
  const barHeight = 40
  const gap = 15
</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  {#each data as d, i}
    <rect
      x={0}
      y={i * (barHeight + gap) + 20}
      width={d.temp * 4}
      height={barHeight}
      fill="steelblue"
    />
    <text
      x={d.temp * 4 + 8}
      y={i * (barHeight + gap) + 20 + barHeight / 2 + 5}
      font-size="14"
    >{d.city}</text>
```

```
    {/each}  
</svg>
```

`width={d.temp * 4}` is the channel. The multiplier is a manual scale factor that maps data values to pixel widths.

Scatter plot

Two quantitative channels: x position and y position. Each row becomes a point.

None

```
<script>  
  const data = [  
    { city: "New York",    income: 67000, rent: 1800 },  
    { city: "Philadelphia", income: 47000, rent: 1200 },  
    { city: "Chicago",     income: 58000, rent: 1300 },  
    { city: "Los Angeles", income: 65000, rent: 1900 },  
    { city: "Houston",     income: 52000, rent: 1050 },  
    { city: "Phoenix",     income: 51000, rent: 1100 },  
    { city: "Seattle",     income: 93000, rent: 2000 },  
    { city: "Miami",       income: 45000, rent: 1600 }  
  ]  
  
  const width = 600  
  const height = 400  
  const margin = 60  
  
  // income range: ~40k–100k, mapped to x axis  
  const xScale = income => ((income - 40000) / 60000) * (width -  
margin * 2) + margin  
  
  // rent range: ~1000–2000, mapped to y axis (inverted: higher  
rent = higher on canvas)  
  const yScale = rent => height - margin - ((rent - 1000) / 1000) *  
(height - margin * 2)
```



```

</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  {#each data as d}
    <circle
      cx={xScale(d.income)}
      cy={yScale(d.rent)}
      r="8"
      fill="steelblue"
      opacity="0.7"
    />
  {/each}
</svg>

```

The scale functions handle the translation from data values to pixel positions. The pattern is: subtract the minimum, divide by the range, multiply by the available pixels.

Line chart

A line chart connects points in sequence. Instead of one element per row, you build a single path from all the rows.

None

```

<script>
  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 },
    { month: "Jul", temp: 88 },
    { month: "Aug", temp: 86 },
    { month: "Sep", temp: 78 },
  ]

```

```

    { month: "Oct", temp: 67 },
    { month: "Nov", temp: 55 },
    { month: "Dec", temp: 44 }
  ]

  const width = 600
  const height = 400
  const margin = 60

  const xScale = i => margin + (i / (data.length - 1)) * (width -
margin * 2)
  const yScale = temp => height - margin - (temp / 100) * (height -
margin * 2)

  // build the path string by joining all points
  const points = data.map((d, i) =>
`${xScale(i)},${yScale(d.temp)}`).join(" ")
</script>

<svg viewBox="0 0 {width} {height}" {width} {height} style="border:
1px solid #ccc">
  <!-- the line -->
  <polyline
    points={points}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />

  <!-- a dot at each data point -->
  {#each data as d, i}
    <circle
      cx={xScale(i)}
      cy={yScale(d.temp)}
      r="4"
      fill="steelblue"

```

```
    />  
  {/each}  
</svg>
```

The key difference from a column chart: instead of one element per row, you're building a single `<polyline>` from all the rows. The `points` string is a list of `x,y` coordinates separated by spaces.

The scale problem

You may have noticed magic numbers scattered through these examples: `* 4, / 60000, / 100`. These are manual scale calculations. They work for this specific data but break the moment the data changes range. D3 provides scale functions that handle this automatically, which is covered in the Week 2 materials.